Park & Ride Sri Lanka — Statement of Work

Oct 16, 2026 · @Padma

1. Project overview

Park & Ride Sri Lanka is a web app that lets a commuter park a vehicle at a Metro bus terminal, book a seat on a live-tracked Metro bus, and board and alight by QR scan — built at least 85% in Jac (Jaseci Labs) with Google Antigravity as the coding agent.

Today a commuter driving to a terminal cannot see free parking, cannot reserve a slot, and cannot tell how full the next bus is. The app joins those three steps into one flow and gives operators a live view of every bus, every stop and every passenger count.

Terminals (park-and-ride hubs): Kesbewa, Kadawatha, Pettah, Makumbura.

Routes in scope:

Route	Major stops served
Makumbura → Pettah	Kottawa, Battaramulla, Borella, Town Hall
Makumbura → Kadawatha	Apeksha Hospital, Nawinna, Narahenpita, Kelaniya Campus
Kesbewa → Pettah	Piliyandala, Werahera, Boralesgamuwa, Kohuwala, Havelock

Two user groups:

- **Passengers** — choose *Park & Ride* or *Get into Bus* on the landing pop-up, book parking or a seat, and scan QR codes at boarding and alighting.
- **Admins / operators** — watch live buses on a Sri Lanka map, open each bus's account (driver, conductor, next stop, expected alighting, stop QR), and get alerts when counts don't match.

The build is a competition-grade prototype: real logic end to end, with simulated bus GPS and a demo time control so judges can watch a bus move stop to stop.

0. Today's build: one-day plan

The whole project is built today, Saturday 17 Oct 2026, by three developers working in parallel, each on their own laptop. This section overrides the rest of the SOW: build only what is marked **Today**, and treat everything else as the roadmap after the hackathon.

What we build today and what we cut

Area	Today (must work in the demo)	Cut for today
Setup	Jac full-stack project, seed data, two seeded logins (passenger, admin)	Sign-up, roles UI, cloud deployment (run on a laptop)
Parking	Landing pop-up, slot grid for 1 terminal fully (Makumbura) and counts for the other 3, return-time zone rule (front / middle / rear), book slot, parking QR, *Mark vehicle as parked*	Full scoring formula, 30-min hold, extend-return, byLLM reasons
EV	"Is it an electric vehicle?" → EV charging slots (type `E`)	Charger power, charge estimate, waitlist
3D	Simple three.js yard with box cars and the booked slot glowing — *only after the passenger flow works end to end*, else a 2D grid	Detailed models, 3D buses
Bus map	Own SVG map from the road graph, 9 simulated buses moving, tap a bus → seats left → book pickup and drop	Seat-left forecast, Start-from-here, timetables
Scans	Stop QR on the admin next-stop panel, board and get-off scans with `scan_id` dedupe, walk-in destination prompt, auto-cancel at departure, mismatch alert	Pass expiry windows, passenger "Did you get off?" prompt
Admin	Home map + bus list, bus account (driver, conductor), next-stop panel (ETA, getting off, bookings, QR), scan and alert feeds	Parking and charger tab, reports

Area	Today (must work in the demo)	Cut for today
Traffic	Port the traffic prototype: road levels, presets, reroute rule, driver view with turn instruction	Ripple effects, rain mode
Incidents	One *Report incident* button on the driver view that closes the current road and reroutes buses	Admin confirm and expiry, passenger reports
Arrival alert	Banner + vibration 2 stops before the drop stop	Browser notifications, second alert
Demo mode	*Play demo* button that steps through the judge script with captions	Back step, offline phone simulation
Cut entirely today	—	Combined Park & Ride ticket, seat-left forecast, Sinhala and Tamil (English only; one language toggle on the landing screen is a stretch), accessibility extras beyond real buttons and labels, Lanka Metro feed, WebSockets (polling every 2 s)

How each developer runs their own part on localhost

Every developer runs the whole app on their own laptop with `jac start main.jac --watch` and opens `http://localhost:8000`. Nobody waits for anybody: a mock switch lets the passenger and admin screens run on sample data until the real walkers are ready.

- **One API file, one switch.** `components/shared/api.cl.jac` has one function per walker in `docs/contracts.md`. With `USE_MOCK = True` it returns the sample responses in `mock/*.json`; with `USE_MOCK = False` it calls the real walker with `root spawn`. Screens call only these functions, never walkers directly.
- **Each part has its own address.** Passenger app at `/`, admin panel at `/admin`, driver view at `/driver/MB-101`, and Dev A's core console at `/dev/core`: a plain page with one button per walker that shows the JSON it returns.
- **Same data for everyone.** `jac run core/seed/seed.jac` (or the *Reset* button on `/dev/core`) loads the same seed on every laptop, so screens and bugs match.

- **Switch off mocks part by part.** When Dev A says a walker is real, B and C set its line in `api.cl.jac` to real and re-check their screens.

Order of work

The work is split into five milestones. A milestone is done when every developer has ticked their local check for it; then everyone merges to `main`, pulls, and runs the integration check together.

Milestone	Dev A — Claude Code	Dev B — Antigravity	Dev C — Antigravity	Integration check (everyone, on `main`)
0. Foundation	Repo, AGENTS.md, models, seed, `contracts.md`, `mock/*.json`, `api.cl.jac` with the mock switch, stubs for every walker, `/dev/core` page, CI	Install Jaseci and Antigravity rules, open `/` and see the starter page	Same, open `/admin`	`jac start` runs on all three laptops; `/`, `/admin`, `/driver/MB-101` and `/dev/core` all open
1. Parking	Real parking walkers: zone rule, EV slots, book, signed QR, mark parked; tests	Landing pop-up, login, Park & Ride flow with EV question, slot grid, QR screen	Bus list, bus account, next-stop panel with stop QR, scan and alert feeds (on mocks)	Book a car slot at Makumbura on `/`, mark parked, see the slot turn occupied on `/dev/core`
2. Bus journey	`book_seat`, per-segment seat counting, `process_scan` with dedupe, walk-ins, auto-cancel, mismatch; tests	Shared `CityMap`, bus map, bus card, seat booking, QR scanner, trip view	Admin Home map with `CityMap`, click bus to open account	Book a seat on `/`, scan the stop QR shown on `/admin` with a phone, see the scan in the admin feed

Milestone	Dev A — Claude Code	Dev B — Antigravity	Dev C — Antigravity	Integration check (everyone, on `main`)
3. Traffic and driver	Port the traffic prototype: `plan_route`, `DemoClock`, `move_bus`, `set_traffic`, `report_incident`, arrival flag; port its 7 self-checks as tests	Arrival banner and delay messages on the trip view	Demo console (speed, jump to stop, presets), driver view with turn instruction and *Report incident*	Choose *Crash near Rajagiriya* on `/admin`: `/driver/MB-101` shows the new route and the passenger trip view shows the delay
4. Demo mode	`run_demo` engine calling the real walkers, reset under 5 s	3D yard if the flow is solid, else polish	*Play demo* button and caption bar	Press *Play demo* and watch the full judge story run twice without errors

After milestone 4: feature freeze, bug fixes only, two full rehearsals, a backup screen recording, and a printed stop QR as a fallback.

Check your part on localhost

Dev A — core

- [ ] `jac check` passes on every `.jac` file and `jac test` is all green
- [ ] `/dev/core`: every walker button returns JSON shaped exactly as in `contracts.md`
- [ ] Scanning the same `scan_id` twice returns `duplicate`; a pending booking not scanned at departure becomes `Cancelled`
- [ ] A closed road makes MB-101's route change; a slow road does not
- [ ] `run_demo` plays every step and *Restart* resets in under 5 s
- [ ] `python scripts/jac_share.py` reports at least 85% Jac

Dev B — passenger (`/`)

- [ ] With `USE_MOCK = True`, every screen works with no backend errors in the browser console

- [ ] Park & Ride: car + EV *Needs charging* offers only `E` slots; booking shows a QR; *Mark vehicle as parked* opens the bus map
- [ ] Bus map shows 9 buses moving every 2 s; tapping one shows seats left and books a seat
- [ ] Scanning a test QR shows boarded, then alighted on the second scan; a repeat scan shows "already counted"
- [ ] With `USE_MOCK = False`, the same steps work against Dev A's walkers
- [ ] Works at 390 px wide (browser device mode)

Dev C — admin and driver (`/admin`, `/driver/MB-101`)

- [ ] With `USE_MOCK = True`, every tab loads with sample data
- [ ] Bus account shows a different driver and conductor for each bus
- [ ] Next-stop panel shows stop, ETA, getting off, bookings and a QR that a phone camera can read from 1 m
- [ ] Demo console speed and jump-to-stop change the clock and the next-stop panel
- [ ] A traffic preset or *Report incident* changes the driver's instruction and shows the reroute banner
- [ ] *Play demo* steps through every caption

Rules for today: commit small and often on your own branch (`a/…`, `b/…`, `c/…`), merge to `main` only at milestone ends after the local checks are ticked, and cut any task that is stuck rather than holding up a milestone. Claude Code usage on the Pro plan can run out during a long day, so Dev A should spend it on the walkers, the routing port and the demo engine.

Today's prompts

Paste each as the first message of the session. Dev B and Dev C should use Antigravity's Planning mode and approve the plan before code is written.

Dev A — Claude Code

```
You are the core developer for "Park & Ride Sri Lanka", built today by three
developers on separate laptops. Dev B (Antigravity) builds the passenger app
in pages/ and components/passenger/ plus the shared map
components/shared/CityMap.cl.jac. Dev C (Antigravity) builds the admin panel
and driver view in admin/, driver/ and components/admin/. You own core/,
tests/, scripts/, mock/, .github/, docs/contracts.md,
components/shared/api.cl.jac and the /dev/core page, and you review and merge
```

```
their PRs.

Read AGENTS.md and section 0 of docs/sow.md first. Build only what section 0
marks Today. Work milestone by milestone and stop after each so I can tick
the local checks in section 0.

Stack: Jac full-stack (jac-client), walkers as the API (walker:pub for
passengers, walker:priv for admin), frontend calls via root spawn, polling
every 2 s. No Google Maps, payments or WebSockets. If unsure of Jac syntax,
use docs/jac-reference.md or the jac MCP server; never guess. Run jac check
and jac test before every commit.

MILESTONE 0 — foundation (push this first; the others start from it):
- jac create --use client; entry main.jac with routes / (passenger), /admin,
/driver/:busId, /dev/core.
- core/models: Terminal, Slot (types B, M, C, E; zones F, M, R; codes like
MKB-C-F-07; states free, booked, occupied), Route, Stop, Junction, Road edge
(km, class A/B/E, level 0–3), Bus (plate, capacity 40, driver, conductor),
Trip, StopVisit (qr token, visit number), Booking (pickup, drop,
Pending/OnBoard/Alighted/Cancelled, walk_in), Scan (scan_id), Passenger
(role), Alert, Incident, DemoClock (speed, offset, paused).
- core/seed/seed.jac from the SOW appendix plus the J and ROADS tables of the
traffic prototype linked in SOW section 5; seeded logins passenger and admin.
- docs/contracts.md with exact inputs, outputs and error codes for:
get_terminals, get_slots, recommend_slots, book_slot, mark_parked, get_map,
get_buses, book_seat, process_scan, get_trip, admin_bus_list,
admin_bus_account, admin_next_stop, admin_feed, demo_control, set_traffic,
report_incident, get_driver_view, run_demo, reset_demo.
- mock/<walker>.json with one realistic sample response per walker, and
components/shared/api.cl.jac with one function per walker plus a USE_MOCK
flag per walker (mock JSON when true, root spawn when false).
- A stub for every walker returning seed data, a plain /dev/core page with a
button per walker showing its JSON and a Reset button, scripts/jac_share.py
(fails under 85% Jac) and .github/workflows/ci.yml (jac check, jac test,
jac_share).

MILESTONE 1 — parking (tests first): zone by stay length (under 3 h front, 3–
8 h middle, over 8 h rear), 3 options, EV Needs charging -> E slots only,
Park only -> car slots, book_slot with an HMAC-signed QR, mark_parked.

MILESTONE 2 — bus journey (tests first): book_seat with per-segment seat
counting (SOW 9.2) and no passed-stop pickups; process_scan exactly per the
SOW 9.4 table with scan_id dedupe and walk-ins, safe if two scans arrive at
```

```
once; depart_stop auto-cancel and the mismatch alert (9.5).

MILESTONE 3 — traffic: port the traffic prototype to Jac. plan_route =
Dijkstra on km / (A30, B22, E60 km/h × 1, 0.55, 0.25, closed) × 60. DemoClock
drives everything (speed, jump bus to stop, pause). move_bus: no U-turns mid-
road, switch only if it saves at least 1 min or 10%, wait if no open road.
set_traffic with presets clear, morning rush, crash near Rajagiriya, High
Level Road jam, Makumbura cut off. report_incident closes the bus's current
road and re-plans buses. get_trip sets alert true 2 stops before the drop
stop. Port the prototype's 7 self-checks as jac tests.

MILESTONE 4 — demo: run_demo steps that call the real walkers, each with a
caption: book parking at Makumbura, mark parked, book MB-101 Makumbura to
Borella, board scan, duplicate scan ignored, crash near Rajagiriya and
reroute, arrival alert, get-off scans with one missing, mismatch alert.
Restart resets to seed in under 5 s.

After each milestone: run the Dev A checks in section 0, tell me which
walkers are now real so B and C can switch off their mocks, then review open
PRs against AGENTS.md and contracts.md. Never change a contract without my
OK.
```

Dev B — Antigravity (passenger app)

```
You are building the PASSENGER app of "Park & Ride Sri Lanka", a Jac full-
stack app (jac-client), on my own laptop. Dev A owns the backend walkers in
core/; Dev C builds the admin panel and driver view.

Read AGENTS.md, section 0 of docs/sow.md (build only what is marked Today)
and docs/contracts.md first.

Rules:
- Write only in pages/, components/passenger/ and
components/shared/CityMap.cl.jac. Never edit core/, admin/, driver/ or
api.cl.jac.
- Get all data through the functions in components/shared/api.cl.jac. Keep
USE_MOCK true until Dev A says a walker is real, then switch that walker off
mock and re-test.
- Jac client files (*.cl.jac), has state, polling every 2 s, Tailwind. Mobile
first at 390 px, 44 px touch targets, real buttons and labels. .ts only in
glue/ under 80 lines. No Google Maps.
- Run the app with jac start main.jac --watch and test at
http://localhost:8000/. Run jac check before every commit. Commit small on
```

```
branch b/<task>.
- Look: navy text #0F1B2D, yellow primary buttons #FFC21A, route colours R1
blue, R2 purple, R3 teal; fonts Schibsted Grotesk and IBM Plex Mono; follow
the passenger screens in the design canvas.

Milestone 1 — parking: landing pop-up with Park & Ride and Get into bus;
login for the seeded passenger; terminal picker with free counts; vehicle
type; for cars ask Is it an electric vehicle (Needs charging / Park only /
No); arrive and return-by times; slot grid by zone (free, booked, occupied,
recommended, option); three options with reasons; book -> slot code and QR
(qrcode.react) -> Mark vehicle as parked -> bus map.

Milestone 2 — bus journey: components/shared/CityMap.cl.jac, an SVG map from
get_map (roads grey free, amber slow, red jammed, dark dashed closed; stops;
labels) with bus markers from get_buses every 2 s, rotated by heading,
coloured by route; props onBusClick and highlightBusId (Dev C reuses it). Bus
map page: tap a bus -> route, next stop, ETA, seats left, Confirm; choose
pickup and drop with passed stops hidden; book_seat. Camera scanner (html5-
qrcode in glue/) calling process_scan with a fresh random scan_id; if
walk_in_needs_drop, ask for a destination and rescan. Trip view from
get_trip.

Milestone 3: on the trip view, an alert banner plus navigator.vibrate when
alert is true ("Your stop is 2 stops away"), and delay or reroute messages.

Milestone 4: only if everything works, a simple three.js yard (box cars,
booked slot glowing, drag to rotate); otherwise polish loading, empty and
error states.

After each milestone, go through the Dev B checklist in section 0 on
localhost and report what passes. If a contract field is missing, don't
invent a walker: note it in the PR for Dev A and show a placeholder.
```

Dev C — Antigravity (admin panel and driver view)

```
You are building the ADMIN PANEL and DRIVER VIEW of "Park & Ride Sri Lanka",
a Jac full-stack app (jac-client), on my own laptop. Dev A owns the backend
walkers in core/; Dev B builds the passenger app and the shared map
components/shared/CityMap.cl.jac.

Read AGENTS.md, section 0 of docs/sow.md (build only what is marked Today)
and docs/contracts.md first.
```

```
Rules:
- Write only in admin/, driver/ and components/admin/. Never edit core/,
pages/, CityMap or api.cl.jac; ask in the PR if you need a CityMap prop.
Until CityMap exists, use a grey placeholder box.
- Get all data through the functions in components/shared/api.cl.jac. Keep
USE_MOCK true until Dev A says a walker is real, then switch it off and re-
test. Admin walkers need the seeded admin login.
- Jac client files (*.cl.jac), polling every 2 s, Tailwind. Admin is shown on
a projector: dark theme (background #0B1220, surface #111A29, text #E8EEF6),
large text, high contrast, real buttons and labels. No Google Maps.
- Run the app with jac start main.jac --watch and test at
http://localhost:8000/admin and http://localhost:8000/driver/MB-101. Run jac
check before every commit. Commit small on branch c/<task>.
- Follow the admin screens in the design canvas and the traffic prototype
linked in SOW section 5.

Milestone 0 — admin routes guarded by the admin role, top bar with the demo
clock, tabs Home, Bus account, Demo console; driver view shell at
/driver/:busId.

Milestone 1 — bus list (route colour, next stop, ETA, load, alert dot) -> bus
account (plate, route, driver, conductor, on board / capacity); next-stop
panel (stop, ETA, getting off, bookings to board, walk-ins, seats left after
the stop, stop QR big enough to scan from 1 m); scan feed and alert feed,
newest first.

Milestone 2 — Home tab with CityMap and all buses; clicking a bus opens its
account.

Milestone 3 — demo console: speed 1×, 10×, 30×, jump bus to stop, pause,
traffic presets (Clear, Morning rush, Crash near Rajagiriya, High Level Road
jam, Makumbura cut off). Driver view from get_driver_view: green instruction
bar ("Turn right toward Nawala · 1.3 km"), next stop and ETA, yellow New
route banner with the reason, and a big Report incident button (type:
accident, breakdown, flooding, road works, road closed; severity: slow,
jammed, closed).

Milestone 4 — Play demo button and a caption bar across the bottom of the
admin screen with Pause, Next and Restart, calling run_demo.

After each milestone, go through the Dev C checklist in section 0 on
localhost and report what passes. If a contract field is missing, don't
invent a walker: note it in the PR for Dev A and show a placeholder.
```

2. Objectives and success criteria

The project succeeds when a judge can park, ride, scan in, scan out and see the admin counts reconcile — on one live demo, with at least 85% of hand-written code in Jac.

#	Objective	Measured by
O1	One-language build	⩾ 85% of hand-written lines are `.jac` / `.cl.jac`, counted with `cloc` excluding `node_modules`, `.jac/` build output, lock files and generated code
O2	Real-time parking	Slot grid for each terminal refreshes within 3 s of a booking or release
O3	Smart slot allocation	Every booking returns a recommended slot plus 2 alternatives, ranked by return time
O4	Live bus map	Buses move on the app's own map along all 3 routes; tap a bus to see seats left
O5	Honest seat counts	Seats-left on each bus equals capacity − onboard − confirmed bookings ahead, at every stop
O6	QR lifecycle	Parking QR, booking QR, stop QR boarding and stop QR alighting all work, with duplicate scans ignored
O7	Operator control	Admin sees next stop, ETA, expected alighting, bookings and the stop QR; mismatch alerts fire
O8	Judge-ready demo	A demo console can speed up time, jump a bus to any stop, set traffic on any road, and replay the scenario

How the 85% is kept honest: The map is drawn by Jac client components from the road graph, so it counts as Jac. three.js is the only large npm library, and it is called from Jac client components (`cl` blocks). Only thin wrappers that Jac cannot express — such as a three.js render loop helper — may live in small `.ts` files, capped at roughly 10% of lines. A `cloc` report is committed at every milestone.

3. Scope

The prototype covers the full passenger journey and the operator panel for 4 terminals and 3 routes; payments, real bus hardware and native mobile apps are out.

In scope

- Landing pop-up with two choices: *Park & Ride* and *Get into Bus*.
- Passenger sign-up and login (Jac built-in auth), plus a separate admin role.
- Parking: live slot map per terminal for 3 vehicle types (foot cycle, motorcycle, car), availability check, return-time capture, smart slot recommendation with alternatives, booking, parking QR, three.js 3D view of the yard, *Mark as parked* button.
- Bus tracking: the app's own map of Colombo with the 3 routes drawn, traffic scenarios and live rerouting on the driver view, animated mini bus models, tap-a-bus popup with seats left, confirm selection, pickup and drop selection, *Start from here* at main halts.
- QR flows: stop QR shown on the admin side, passenger scans to confirm boarding and again to alight, unique scan IDs to block duplicates, auto-cancel of unconfirmed bookings.
- Driver-side notification of waiting passenger count at each stop.
- Admin panel: Home map with live buses, per-bus account page, next-stop panel with ETA, alighting count, bookings, stop QR, traffic-aware path, mismatch alerts, demo time controls.
- Seed data: slots, buses, drivers, conductors, stops, timetables.

Added on 17 Oct 2026 (section 8A): arrival alerts, one combined Park & Ride ticket, a seat-left forecast, accessibility, Sinhala and Tamil, EV charging slots, incident reporting and a one-click demo mode.

Out of scope

- Real payments and fares (LMT-GO already handles tap-in/tap-out).
- Physical GPS units or IR passenger counters on buses — GPS is simulated unless LMT grants a data feed.
- Native Android/iOS apps; the web app is responsive and works on phones.
- Routes beyond the three listed.
- Google Maps and real-time traffic feeds. The app draws its own map and simulates traffic (see section 10).

4. Technology stack

One Jac codebase serves both the React-style frontend and the graph backend; Google Maps and three.js are the only major non-Jac libraries, called from Jac client components.

Layer	Choice	Why
Language	Jac (`jaclang`), installed with `pip install jaseci`	Python superset with nodes, edges and walkers — the bus network *is* a graph
Frontend	`jac-client` — `cl { }` blocks / `.cl.jac` files, JSX, `has` state (auto `useState`), React Router via `@jac-client/utils`	Frontend written in Jac; walkers called with `root spawn walker_name()` instead of fetch/axios
Backend API	`walker:pub` / `walker:priv` served by `jac start` (jac-scale FastAPI server); each public walker becomes `POST /walker/<name>`	No hand-written REST layer
Auth	`jacLogin` , `jacSignup` , `jacLogout` , `jacIsLoggedIn` + a `role` field on the user's profile node	Built-in; admin routes wrapped in a protected route
Persistence	Local: SQLite (default). Deploy: `jac start --scale` → MongoDB + Redis on Kubernetes	Same code laptop → cloud
Real-time	Polling walkers every 2 s for the MVP; upgrade to jac-scale WebSocket walkers (`/ws/<walker>`) if time allows	Polling is simplest to demo reliably
AI (optional)	`byLLM` — e.g. `def explain_slot_choice(...) -> str by llm();`	One-line plain-English reason for the slot recommendation
Maps	Own map: a road graph of `Junction` nodes and `Road` edges, drawn as SVG by Jac `cl` components; routing by the `plan_route` walker. Optional OpenStreetMap base layer later	No API key or bill, works offline at the venue, traffic fully controllable for the demo, and it raises the Jac share. Working prototype

Layer	Choice	Why
3D	three.js via `@react-three/fiber` + `@react-three/drei`	3D parking yard and the mini bus models
QR	`qrcode.react` (generate), `html5-qrcode` (camera scan)	Small, browser-only
Styling	Tailwind CSS configured in `jac.toml`	Fast, consistent UI
Coding agent	Google Antigravity (Gemini 3.x; Claude models also selectable) — Editor view + Agent Manager	Plans, writes, runs and browser-tests the code; produces reviewable task lists and walkthroughs

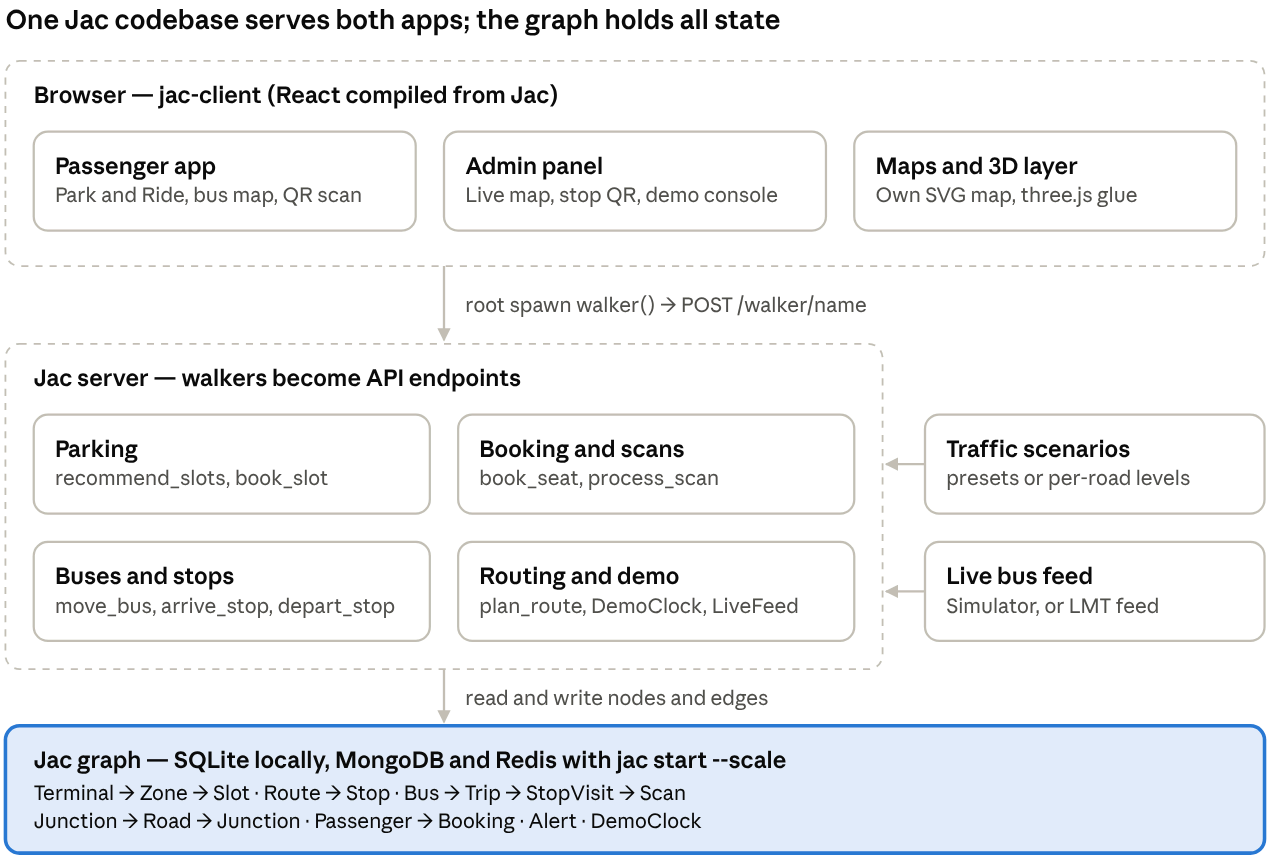
Antigravity setup for Jac. Jac is new enough that agents make syntax mistakes, so give the agent the language before it writes code:

1. Run `jac mcp` and register it as an MCP server in Antigravity. It exposes Jac docs, a pitfalls guide and idiomatic patterns as resources.
2. Put the official condensed language reference (`jaseci-llmdocs`) in the repo under `docs/jac-reference.md` and point the agent's workspace rules at it.
3. Add a workspace rule: *"All new logic goes in `.jac` / `.cl.jac`. Only create `.ts` files for three.js or Maps glue under 80 lines. Run `jac check` after every change."*
4. Use Agent Manager to run two agents in parallel per phase: one on backend walkers, one on client screens.

Commands: `jac create --use client parkride` → `jac start --watch` (HMR) → `jac check` → `jac test` → `jac build` → `jac start --scale` for deployment.

5. System architecture and data model

The browser calls Jac walkers directly; walkers read and write one graph that holds terminals, slots, routes, buses, bookings and scans — so the parking, bus and admin views always agree.



system architecture · 3 layers, 2 external feeds

Only the Maps and 3D layer uses non-Jac code, and only as thin glue; every rule lives in a walker.

Graph data model

Node	Holds	Main edges
`Terminal`	code, name, lat, lng	→ `ParkingZone`; also linked to its `Stop`
`ParkingZone`	zone `F` / `M` / `R`, vehicle type	→ `Slot`
`Slot`	code (`MKB-C-F-07`), state, 3D position	← `ParkingBooking`
`Route`	code, name, colour	→ `Stop` via ordered edge with `seq`
`Stop`	name, lat, lng, is_terminal	← `Booking` as pickup or drop

Node	Holds	Main edges
`Bus`	plate, capacity, driver, conductor, lat, lng, heading	→ `Trip`
`Trip`	route, last departed stop index, status	→ `Booking`, → `StopVisit`
`StopVisit`	stop, visit number, QR token, arrived / departed time	→ `Scan`
`Booking`	pickup, drop, state (Pending, OnBoard, Alighted, Cancelled), walk-in flag	← `Passenger`
`Scan`	unique `scan_id`, type, time	—
`Passenger`	username, role (passenger / admin)	→ `ParkingBooking`, → `Booking`
`ParkingBooking`	slot, arrival, return-by, QR token, state	→ `Slot`
`Alert`	type, message, time, resolved	linked to `Trip` or `Terminal`
`DemoClock`	speed, offset, paused	one per graph, under `root`

Main walkers

Walker	Called by	Does
`seed`	setup	Loads the Appendix data
`get_slots`, `recommend_slots`	passenger	Live slot states; ranked slots (9.1)
`book_slot`, `mark_parked`, `release_slot`	passenger	Parking lifecycle and QR
`get_buses`	passenger, admin	Positions, next stop, seats left
`book_seat`	passenger	Segment check (9.2), creates `Booking`
`process_scan`	passenger	Scan rules (9.4)

Walker	Called by	Does
`tick` → `move_bus`	`DemoClock` / `LiveFeed`	Advances buses, fires stop events
`arrive_stop`	system	Creates `StopVisit` and its QR, notifies the driver
`depart_stop`	system	Auto-cancel (9.3), alighting check (9.5)
`admin_bus_account`, `admin_next_stop` (`walker:priv`)	admin	Bus account and next-stop panel data
`demo_control` (`walker:priv`)	admin	Speed, jump to stop, pause, spawn, reset

Road network. The map is the app's own graph: `Junction` nodes (every stop plus major junctions, with real latitude and longitude) joined by `Road` edges (length in km, class, traffic level 0–3, one edge per direction). Two more walkers use it: `plan_route` finds the fastest path between stops, and `set_traffic` (`walker:priv`, admin only) changes a road's level and re-plans the buses that use it. The traffic prototype runs this model on 34 junctions and 53 roads.

Nodes and fields added for section 8A: `TripPass` (combined ticket, links one `ParkingBooking` to one or more `Booking`s), `Incident` (type, severity, road, reporter, confirmed, expires), `Charger` (power in kW, linked to an EV `Slot`), slot type `E`, a `priority` flag on seats, and `Passenger` settings for language, accessibility needs and alert preferences. New walkers: `book_trip_pass`, `forecast_seats`, `report_incident`, `notify_arrival` and `run_demo`.

6. Park & Ride module

The passenger picks a terminal, sees live free slots by vehicle type, enters a return time, and gets a recommended slot plus two alternatives — then a QR code that encodes the exact slot location.

Slot numbering system. Every slot has a code `TERMINAL-TYPE-ZONE-NN`, for example `MKB-C-F-07`.

Part	Values
Terminal	`KSB` Kesbewa · `KDW` Kadawatha · `PTH` Pettah · `MKB` Makumbura
Type	`B` foot cycle · `M` motorcycle · `C` car
Zone	`F` front line (near exit) · `M` middle · `R` rear / corner
NN	Slot number within the zone, counted from the exit

Requirements

ID	Requirement
P-01	Landing pop-up shows *Park & Ride* and *Get into Bus*; choosing *Get into Bus* goes straight to bus tracking
P-02	Terminal picker shows the 4 terminals on the map with free-slot counts per type
P-03	Slot grid shows every slot as free, booked, occupied or held, refreshed every 2 s
P-04	Passenger selects vehicle type, arrival time and expected return time
P-05	System recommends one slot and two alternatives, each with its zone and a one-line reason
P-06	Short stays go to the front line; long stays go to the rear or corner (rules in section 9)
P-07	Passenger can check availability without logging in; booking requires login
P-08	Booking creates a parking QR holding booking ID, slot code, terminal, return time and a signature
P-09	three.js view shows the yard in 3D, the chosen slot glowing, and a walking line from the gate
P-10	A booked slot is held for 30 minutes past the chosen arrival time, then released

ID	Requirement
P-11	On arrival the gate scan (or the passenger) presses *Mark vehicle as parked*; the slot turns occupied and the app moves to bus tracking
P-12	Passenger can extend return time; if the new time crosses a zone boundary, the app suggests a move but never forces one
P-13	On return, scanning the parking QR again releases the slot

7. Get into Bus module

The passenger sees every Metro bus moving on a Google Map, taps one to see seats left, books a pickup-to-drop seat, and confirms by scanning the stop QR when boarding and again when alighting.

Requirements

ID	Requirement
B-01	Full-screen map of the Colombo metro area, drawn by the app from its own road graph, with the 3 routes drawn as coloured polylines and every stop as a marker
B-02	Each bus is a small 3D bus model (three.js overlay) that glides between GPS points and turns to its heading, like ride-hailing apps
B-03	Tapping a bus opens a card: route, plate number, next stop, ETA to the passenger's stop, seats left, and *Confirm this bus*
B-04	Every booking needs a pickup stop and a drop stop on that bus's route, pickup before drop
B-05	A stop the bus has already passed cannot be chosen as pickup; the app hides it and explains why

ID	Requirement
B-06	At the 4 main terminals, the passenger can tap the terminal and choose *Start from here*, which pre-fills pickup
B-07	Seats left are calculated per segment, so a booking only succeeds if a seat is free for every stop between pickup and drop
B-08	On confirm, the driver's screen gets a notification with the total number of passengers waiting at that stop; it updates live as bookings change
B-09	When the bus reaches a stop, the admin/bus screen shows that stop's QR code for this visit
B-10	**Boarding scan:** passenger scans the stop QR; a booked passenger is confirmed; a passenger with no booking is a walk-in and must pick a destination before the scan completes
B-11	**Alighting scan:** a second scan of a later stop's QR marks the passenger as off the bus and frees the seat
B-12	Every scan sends a unique scan ID; a repeated scan ID, or a scan from a passenger already alighted, is ignored and logged
B-13	A booking not confirmed by the time the bus leaves its pickup stop is cancelled, the seat is freed and the driver count drops
B-14	If the number who alight at a stop differs from the number whose destination was that stop, the admin panel shows an alert listing who is missing or extra
B-15	Passenger sees their trip card: bus, seat status, stops remaining, and an *approaching your stop* banner 2 stops before drop

Why the stop QR changes each visit. The QR shown at a stop encodes bus ID, stop ID, visit number and a short expiry. A photo of yesterday's QR, or of the QR at a different stop, is rejected — so a scan proves the passenger is really at that stop with that bus.

8. Admin panel

The admin opens Home to a live Sri Lanka map of every bus, clicks a bus to open its account, and sees the next stop, ETA, expected alighting, bookings and that stop's QR — with a demo console to drive the scenario for judges.

Requirements

ID	Requirement
A-01	Admin login with role check; passengers cannot open `/admin` routes
A-02	**Home tab:** the app's own map, zoomed to Colombo, all buses live, routes coloured, 4 terminals with parking occupancy badges
A-03	Home side panel lists buses with route, next stop, onboard / capacity and an alert dot
A-04	Clicking a bus opens its **bus account**: plate number, route, driver name, conductor name (unique per bus), today's trips
A-05	Bus account map shows live position and the fastest path to the next stop, recomputed by plan_route whenever traffic changes; the driver's screen shows the same route with turn-by-turn directions
A-06	**Next-stop panel:** stop name, ETA in minutes, people getting off, bookings waiting to board, walk-ins so far, seats left after the stop
A-07	The panel shows the generated stop QR for this bus at this stop, large enough to be scanned from a phone in the demo

ID	Requirement
A-08	Live scan feed: each scan with time, masked passenger ID, scan ID, type (board / alight / walk-in / duplicate ignored)
A-09	Alert feed: alighting mismatch, auto-cancelled bookings, duplicate scans, parking overstay
A-10	**Parking tab:** per-terminal slot grid with the same 3D view as passengers, plus overstay list
A-11	**Demo console:** time-speed slider (1×, 5×, 10×, 30×), *Jump to stop* dropdown, *Pause / Resume*, *Spawn test passenger*, *Reset scenario*
A-12	The demo console moves the simulated clock and bus positions only; every count, QR and alert on screen is recalculated by the real walkers, so judges see the real logic
A-13	Changing the demo clock or jumping to a stop makes the next-stop panel, the QR and the counts change within 2 s

Demo console detail. The simulated clock is a single `DemoClock` node holding `speed`, `offset` and `paused`. Every walker reads time through `now()` on that node instead of the system clock. *Jump to stop* sets the bus's segment progress to that stop and fires the same arrival walker the real GPS feed would.

Traffic scenarios and driver rerouting. The demo console also sets traffic: presets (Morning rush, Crash near Rajagiriya, High Level Road jam, Makumbura cut off, Clear roads) or any single road as free, slow, jammed or closed. Within 2 s every affected bus is re-planned, and the driver view either shows a new route with a banner ("Rajagiriya – Battaramulla closed · saves 6 min") or keeps its route and shows the new ETA. Each decision is logged on the admin panel as *Rerouted*, *Kept route*, *Waiting* or *Moving again*.

8A. Added features

Eight features join the scope as F-01 to F-30: arrival alerts, one combined Park & Ride ticket, a seat-left forecast, accessibility, Sinhala and Tamil, EV charging slots, incident reporting and a one-click demo mode. Today (section 0) only EV slots, incident reporting,

a simple arrival banner and demo mode are built; the rest follows after the hackathon.

Arrival alert

ID	Requirement
F-01	Two stops before the drop stop, the passenger gets an alert: in-app banner, vibration and sound, plus a browser notification if they allowed it
F-02	A second alert fires on arrival at the drop stop: "Get off now and scan the stop QR"
F-03	Alert timing follows the live ETA from `plan_route`, so a reroute or incident moves the alert with it
F-04	If no alighting scan arrives 2 minutes after the bus leaves the drop stop, the app asks "Did you get off?" and the answer feeds the mismatch check (9.5)

Combined Park & Ride ticket

ID	Requirement
F-05	One booking flow books the parking slot and the bus seat together and creates one `TripPass` with one QR
F-06	The pass QR holds pass ID, slot code, booking ID(s), return-by time and a signature; the parking gate and the conductor both read it
F-07	The pass suggests a return bus that arrives before the return-by time, and can book it in one tap
F-08	If the return bus is late because of a logged incident, the parking return time extends automatically and the pass says so
F-09	The pass shows an indicative total (parking + fare) as [PRICE]; no payment is taken in this prototype

Seat-left forecast

The forecast is the seats free on the bus when it reaches the passenger's pickup stop, not the seats free now.

```
\text{forecast} = \text{capacity} - \text{on board} - \text{boarding before pickup} + \text{getting off before pickup} - \text{expected walk-ins}
```

ID	Requirement
F-10	The bus card shows "About 8 seats when it reaches Kottawa" next to the seats free now
F-11	Expected walk-ins per stop = average walk-ins over the last 10 visits to that stop (seed default 2)
F-12	A confidence label: High under 10 minutes away, Medium 10–20 minutes, Low beyond 20 minutes

Accessibility

ID	Requirement
F-13	Accessible parking: wider slots in the front line nearest the gate (2 per terminal for cars), offered first when the passenger ticks "Needs an accessible slot"
F-14	Priority seats: 4 per bus held for passengers who mark the need; the seat counts and forecast keep them separate
F-15	The whole app meets WCAG 2.2 AA: 4.5:1 text contrast, 44 px touch targets, labels for screen readers, full keyboard use
F-16	Settings: larger text, high-contrast mode, and reduced motion (turns off 3D animation and map rotation)
F-17	Optional spoken next-stop announcements using the browser's speech feature

Sinhala and Tamil

ID	Requirement
F-18	Language switch English / සිංහල / தமிழ் on the landing pop-up and in settings, remembered per user
F-19	Every screen, alert, banner and QR-scan message comes from one translation file per language; stop and terminal names are stored in all three languages in the seed data
F-20	Fonts: Noto Sans Sinhala and Noto Sans Tamil. Translations can be drafted with `by llm()` but are checked by a native speaker before the demo

Example stop names to verify: Pettah – පිටකොටුව – [illegible]; Borella – බොරැල්ල – [illegible].

EV charging slots

ID	Requirement
F-21	When the vehicle is a car, the app asks "Is it an electric vehicle?" If yes, the passenger picks *Needs charging* or *Park only*
F-22	*Needs charging* goes to EV slots (type `E`, for example `MKB-E-F-02`) with a charger of 7 kW, 22 kW or 50 kW; the app shows the expected charge by the return time. *Park only* EVs use normal car slots
F-23	If every EV slot is taken, the app offers a normal car slot and a place on the charger waitlist, and moves the booking when a charger frees up

In the 3D yard, charging slots glow green while charging. The admin parking tab shows charger use per terminal and flags cars that are fully charged but still parked.

Incident reporting

ID	Requirement
F-24	From the driver view, a driver reports an incident in two taps: accident, breakdown, flooding, road works or road closed, with severity slow, jammed or closed. The location is the bus's current road
F-25	Each report creates an `Incident` node linked to that `Road`, applies `set_traffic` at once and re-plans every affected bus
F-26	Passengers on affected buses see "Delay: accident on Rajagiriya – Battaramulla, ETA +6 min"
F-27	The admin confirms, edits or clears each incident. Unconfirmed incidents expire after 30 minutes; a second report on the same road raises its confidence. Passenger reports are optional and need admin approval

One-click demo mode

ID	Requirement
F-28	A *Play demo* button on the admin panel runs the full judge story on the `DemoClock`: park, book, board scan, crash incident, reroute, arrival alert, get-off scan, mismatch alert
F-29	A caption bar explains each step for the judges, with Pause, Back, Next and Restart controls; Restart resets to seed data in under 5 seconds
F-30	The demo script is a list of steps that call the same walkers as real use, so it shows the real logic. It runs offline with simulated scans, and a real phone scan can join at any step

9. Key algorithms and business rules

Five rules carry the logic: zone-by-return-time parking, per-segment seat counting, a booking state machine, one-shot scan processing, and an alighting check at every departure.

9.1 Parking slot allocation

Expected stay	Zone	Reason
Up to 3 h	`F` front line	Leaves first, so never boxed in
3 h to 8 h	`M` middle	Typical work day
Over 8 h or overnight	`R` rear / corner	Out of the way of short-stay traffic

Among free slots of the right vehicle type, each slot is scored and the top three are offered:

```
\text{score} = 100\cdot\text{zoneMatch} - 5\cdot\text{exitRank} -
20\cdot\text{blocksEarlierLeaver} + 10\cdot\text{nextToSameType}
```

The best slot is the recommendation; alternative 1 is the next best in the same zone; alternative 2 is the best in the neighbouring zone. A `by llm()` function may turn the score into a one-line reason, with a fixed template as fallback.

9.2 Seats left, per segment

A route is an ordered list of stops. For each segment *s* (between stop *s* and stop *s+1*), occupancy counts riders on board who have not reached their drop, plus bookings whose pickup ⩽ *s* < drop.

```
\text{seatsLeft}(s) = \text{capacity} - \text{occupancy}(s)
```

A booking from pickup *p* to drop *d* is accepted only if occupancy is below capacity on every segment from *p* to *d − 1*. The tap-a-bus card shows seats left on the passenger's own segment.

9.3 Booking rules

- Pickup must be a stop the bus has not yet left (index greater than the last departed stop).
- When the bus departs stop *s*, every booking with pickup *s* that is still *Pending* becomes *Cancelled*; the seat frees and the driver count drops.
- The driver's waiting count for stop *s* = bookings with pickup *s* still *Pending*, pushed whenever it changes.

9.4 Scan processing (one rule table)

Each scan carries `scan_id` (UUID made on the phone), the passenger's session and the stop-QR token. The `process_scan` walker checks the token's signature and expiry, then:

Passenger state on this bus	Result
`scan_id` already seen	Ignored, logged as duplicate
No booking, not on board	Walk-in: must choose a destination, then *On board*
Booking *Pending*, this stop is pickup	Booking *Confirmed* → *On board*
On board, this stop is after pickup	*Alighted*, seat freed
Already *Alighted*	Ignored
Booking for a different bus or stop	Rejected with a message

9.5 Alighting check

When the bus departs stop *s*, the `depart_stop` walker compares *expected* (on-board riders whose drop is *s*) with *actual* (alight scans at *s*). Any difference raises a `Mismatch` alert naming the extra and the missing riders. Riders who stay on past their drop are kept on board and flagged.

9.6 Traffic-aware routing

Each road's travel time comes from its length, its class speed and its traffic level; the fastest path is Dijkstra on those minutes.

```
\text{minutes} = \frac{\text{km}}{\text{speed}_{\text{class}}} \times
f_{\text{level}}} \times 60,\qquad f = 1,\ 0.55,\ 0.25,\ \text{closed}
```

Rule	Value
Clear-road bus speed	Main road 30 km/h, local road 22 km/h, expressway 60 km/h
When to re-check	On every traffic change, and every 15 simulated seconds
Switch only if	The new route saves at least 1 min or 10%, so drivers aren't sent back and forth
Bus already on a road	Finishes that road; the new route starts at its far end (no U-turns mid-road)
Stops	Rerouting only changes roads between two stops; the bus still serves every stop in order
No open road	The bus waits at the last junction and moves on when a road reopens

Sketch of the Jac model (verify with `jac check`, since edge-query syntax varies between Jac versions):

```
node Junction { has code: str; has name: str; has lat: float; has lng: float;
}

edge Road {
    has a: str; has b: str;      # junction codes at each end
    has km: float; has kmh: float;
    has level: int = 0;          # 0 free, 1 slow, 2 jammed, 3 closed
}

# seeding one direction of a road
bor +>: Road(a="BOR", b="RAJ", km=2.3, kmh=30.0) :+> raj;
```

10. Data sources and integration plan

Live bus data comes through one swappable adapter: a built-in simulator by default, and the Lanka Metro feed only if LMT gives permission — Google Maps is used through its official APIs, never scraped.

Data	Source	How it is used	Constraint
Live bus positions	`SimulatorFeed` (default) — buses advance along each route on the `DemoClock`	Drives the map, ETAs and stop arrivals	Fully under our control for the demo
Live bus positions (optional)	`LmtFeed` — Lanka Metro Transit, only with written permission	Same interface as the simulator	SmartMetro publicly tracks routes CM01 and CM02; no public API is documented, so ask LMT before pulling data
Route lines and traffic-aware ETA	Own road graph (`Junction`, `Road`) with traffic levels set by scenario presets or the admin	Polylines on the map, best path to next stop, ETA	Traffic is simulated; road lengths are simplified, not survey-accurate
Stop and terminal coordinates	Entered by the team (picked on site or from public stop lists) and stored as seed data	Stop markers, geofence for *bus has arrived*	If coordinates come from OpenStreetMap, credit © OpenStreetMap contributors
Timetables and headways	Sample values in seed data	Simulator spacing between buses	Replace with LMT data if granted
Parking slots	Sample values (Appendix)	Slot grids and 3D yard	Invented for the prototype

Lanka Metro context. The Ministry of Transport announced a Metro Bus Digital Platform pilot of 10 buses in April 2026 on the Makumbura–Kadawatha and Makumbura–Pettah routes, with real-time fleet monitoring and tap-in/tap-out payments (notice). Two of our three routes match that pilot, which makes LMT a natural partner. Kesbewa–Pettah is simulated.

Asking LMT for data. Send a short email to the LMT support address listed on lankametro.lk describing the student/competition project, the read-only data needed (bus ID, lat/lng, heading, timestamp, route), and an offer to show them the demo. Build against the `LiveFeed` interface so the answer never blocks the project.

Google Maps is optional, not needed. Its official APIs are allowed (only scraping is not), but the build no longer depends on them. If it is added later: no scraping or caching of Maps content, never a Google Map and a non-Google map on the same screen, show Google's attribution, and restrict the API key.

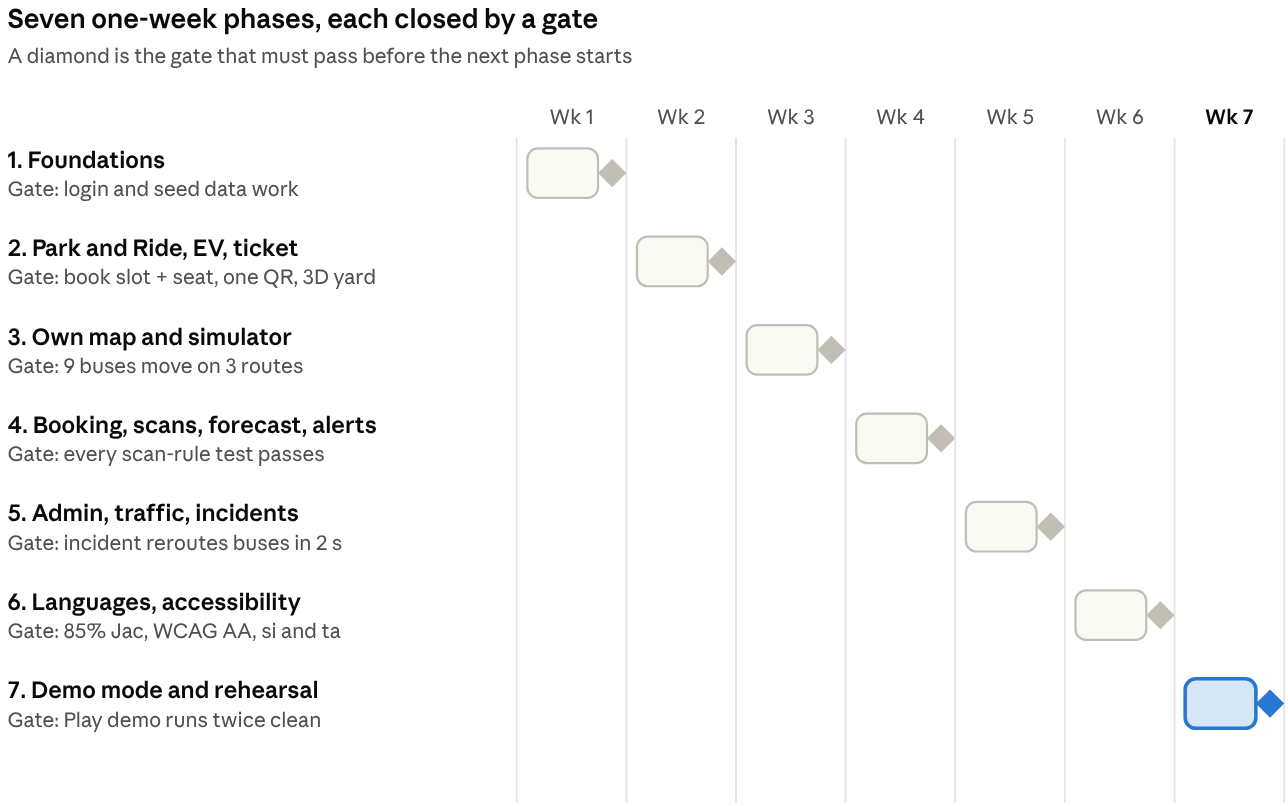
11. Deliverables

Nine deliverables, all in one Git repository plus a hosted demo.

#	Deliverable	Contents
D1	Source code	Jac project (`jac.toml` , `main.jac` , `walkers/` , `nodes/` , `pages/*.cl.jac` , `components/*.cl.jac`), small `glue/*.ts` for three.js and Maps overlay
D2	Seed data	Terminals, 3 routes with ordered stops, 9 buses with driver and conductor names, parking slots per terminal
D3	Passenger app	Landing pop-up, Park & Ride flow with 3D yard and QR, bus map with booking and scanning, trip card
D4	Admin panel	Home map, bus account, next-stop panel with stop QR, scan feed, alerts, parking tab, demo console
D5	Live-feed adapter	`SimulatorFeed` and an `LmtFeed` stub behind one `LiveFeed` interface
D6	Tests	`jac test` suites for allocation, seat counting, scan rules, auto-cancel and mismatch
D7	Jac-share report	`cloc` output at each milestone showing ≥ 85% Jac
D8	Deployment	Hosted URL (`jac start --scale` or a single VM), environment set-up notes, restricted Maps API key
D9	Demo kit	5-minute judge script, two printed phone-scan QR walkthroughs, a backup screen recording

12. After the hackathon: 7-week roadmap

Today's one-day build is in section 0. For the full product after the hackathon, seven one-week phases take the app from an empty Jac project to a rehearsed judge demo; each phase ends at a gate that must pass before the next starts. The plan assumes one developer working with Antigravity agents; move the weeks to fit your competition date.



delivery roadmap · 6 phases, 6 gates

Parking comes before buses because it is self-contained and gives an early demo; the scan rules in phase 4 are the riskiest logic, so they get a full week and their own tests.

Antigravity prompts per phase

Paste each prompt into Antigravity's Agent Manager at the start of the phase. Review the agent's implementation plan before letting it write code, and run `jac check` and `jac test` before accepting.

1. **Foundations** — *"Read docs/jac-reference.md. Create a Jac full-stack project with `jac create --use client parkride`. Add the graph model from the SOW section 5 as Jac nodes and edges in `nodes/`. Write a `seed` walker that loads the Appendix data. Add signup, login and an `admin` role using jacSignup/jacLogin. All code in .jac or .cl.jac."*

2. **Parking** — *"Implement walkers `get_slots`, `recommend_slots`, `book_slot`, `mark_parked`, `release_slot` following SOW section 9.1. Build the Park & Ride pages in .cl.jac with Tailwind: terminal picker, slot grid, return-time form, three recommendation cards, QR screen with qrcode.react. Add a three.js yard view via @react-three/fiber that highlights the booked slot."*
3. **Bus map and simulator** — *"Seed the road graph (Junction nodes, Road edges in both directions) from the appendix. Draw it as an SVG map in .cl.jac with the 3 routes and all stops; no Google Maps. Implement plan_route (Dijkstra on travel minutes) as in SOW 9.6. Implement a `SimulatorFeed` that moves 9 buses along their routes using a DemoClock node. Render each bus as a small three.js bus model in a WebGL overlay that rotates to its heading. Tap a bus to open the bus card from SOW B-03."*
4. **Booking and QR scans** — *"Implement per-segment seat counting (SOW 9.2), `book_seat` with pickup/drop validation, `process_scan` with the rule table in SOW 9.4, `depart_stop` with auto-cancel and the alighting check (SOW 9.3, 9.5). Add the camera scanner with html5-qrcode. Write jac tests for every row of the scan table."*
5. **Admin panel and demo console** — *"Build /admin routes protected by role: Home map, bus list, bus account page with driver and conductor, next-stop panel with ETA from plan_route, stop QR, scan feed, alerts, parking tab. Add the demo console: speed slider, jump to stop, pause, spawn test passenger, reset, traffic presets and per-road traffic levels via a set_traffic walker. The driver view must reroute within 2 s."*
6. **Hardening and demo** — *"Run cloc and report the Jac share; move any logic out of .ts files into Jac until it is at least 85%. Fix every failing test. Use the browser agent to click through the 5-minute judge script and record a walkthrough."*

Fitting in the added features

The section 8A features add about one week, so the plan becomes 7 weeks. Each goes into the phase whose code it touches:

Phase	Features added
2. Park & Ride	EV slots and chargers (F-21–F-23), accessible slots (F-13), combined ticket (F-05–F-09)
4. Booking and QR scans	Seat-left forecast (F-10–F-12), priority seats (F-14), arrival alerts (F-01–F-04)
5. Admin panel	Incident reporting (F-24–F-27)

Phase	Features added
6. Hardening	Accessibility pass (F-15–F-17), Sinhala and Tamil (F-18–F-20)
7. Demo mode (new week)	One-click demo (F-28–F-30), rehearsals

Extra Antigravity prompts:

1. **EV and combined ticket** — *"Add slot type E with a Charger node (7, 22 or 50 kW). When a car is booked, ask if it is an EV and whether it needs charging, following SOW F-21 to F-23. Then add a TripPass node and book_trip_pass walker that books parking and seat together and returns one signed QR (F-05 to F-09)."*
2. **Forecast and alerts** — *"Implement forecast_seats with the SOW 8A formula and confidence labels, and notify_arrival that fires 2 stops before and at the drop stop using plan_route ETAs (F-01 to F-04, F-10 to F-12)."*
3. **Incidents** — *"Add a two-tap Report incident button to the driver view. report_incident creates an Incident linked to the current Road, calls set_traffic and re-plans buses; the admin can confirm, edit or clear, and unconfirmed incidents expire after 30 minutes (F-24 to F-27)."*
4. **Languages and accessibility** — *"Move every UI string into en, si and ta translation files with a language switch (F-18 to F-20). Then run an accessibility pass to WCAG 2.2 AA with text size, high contrast and reduced motion settings (F-15 to F-17)."*
5. **Demo mode** — *"Add run_demo: a Play demo button that steps through the judge script in section 13 on the DemoClock with captions and Pause, Back, Next, Restart. Each step must call the real walkers (F-28 to F-30)."*

13. Testing and demo plan

Every business rule has a `jac test` case, and the judge demo is a scripted 5-minute run on two screens: a phone as the passenger, a laptop as the admin.

Key test cases

Area	Test	Expected
Allocation	Car, return in 2 h, at Makumbura	Recommended slot in zone `F`, two alternatives

Area	Test	Expected
Allocation	Car, return in 10 h	Recommended slot in zone `R`
Seats	Bus at capacity on segment Kottawa→Battaramulla	Booking Kottawa→Borella refused; Borella→Town Hall accepted if free
Passed stop	Bus has left Kottawa	Kottawa not offered as pickup
Scan	Same `scan_id` sent twice	Second ignored, logged as duplicate
Scan	No booking, scan at Battaramulla	Walk-in prompt for destination
Scan	Already alighted, scans again	Ignored
Auto-cancel	Booking pending, bus departs pickup	Booking cancelled, driver count −1, seat freed
Mismatch	3 riders due off at Borella, 2 scan	Alert naming the missing rider
Demo clock	Jump bus to Town Hall	Next-stop panel, QR and counts change within 2 s
Traffic	Bus on Borella→Rajagiriya, then Rajagiriya – Battaramulla jammed or closed	Reroutes via Nawala from the end of its current road; driver view turns right; log says why
Traffic	Same road only slow	Keeps its route (saving under 1 min or 10%); ETA rises
Traffic	Both roads into Makumbura closed	Bus waits at the last junction, then moves on when a road reopens
Arrival alert	Booking Makumbura → Borella, bus leaves Kottawa	Alert fires 2 stops before Borella, then again at Borella
Combined ticket	Book slot + seat in one flow	One TripPass, one QR accepted at the gate and by the conductor
Forecast	40 seats, 30 on board, 5 boarding and 4 getting off before pickup, 2 walk-ins	Forecast 7 seats
EV	EV needs charging, all EV slots taken	Normal car slot offered plus charger waitlist
Incident	Driver reports accident (closed) on the current road	Road closes, affected buses reroute, passengers see the delay

Area	Test	Expected
Language	Switch to Tamil mid-trip	All screens and alerts in Tamil, no English strings left
Demo mode	Play demo, then Restart	All steps run without a phone; reset in under 5 s

5-minute judge script

1. Phone: open the app, choose *Park & Ride*, pick Makumbura, car, return in 2 h. Show the front-line recommendation, the 3D yard and the parking QR.
2. Tap *Mark vehicle as parked*; the app jumps to the bus map with buses moving.
3. Tap a Makumbura→Pettah bus: show seats left; book Makumbura → Borella. Laptop: driver count at Makumbura goes up by 1.
4. Laptop demo console: jump the bus to Makumbura. The stop QR appears. Phone scans it — *Confirmed, on board*.
5. Scan the same QR again — *Already on board, ignored* — and point at the scan feed.
6. Spawn three test passengers due off at Borella; set speed to 10× and jump to Borella. Two of them scan off, and the phone scans the Borella QR — *Alighted*; seats left goes up.
7. Let the bus depart Borella: 4 were due off but 3 scanned, so the mismatch alert names the missing passenger.
8. Close on the `cloc` slide: share of code in Jac.

Traffic step (add 1 minute). Follow MB-101 between Borella and Battaramulla, then choose *Crash near Rajagiriya* on the demo console. The driver view switches to "Turn right toward Nawala", the banner shows why, and the admin log records the reroute. Then set the same road to *Slow* only, to show the bus keeping its route when no road is faster.

14. Risks, assumptions and acceptance

The biggest risk is Jac being new to the coding agent; the biggest dependency is Google Maps billing — both have a fallback.

Risks

Risk	Effect	Mitigation
Agent writes invalid or outdated Jac	Slow progress, broken builds	`jac mcp` + `jaseci-llmdocs` in context, workspace rule to run `jac check`, small reviewed steps
jac-client gaps for three.js or Maps	Logic drifts into `.ts`, Jac share falls	Keep `.ts` to thin glue; all state and logic stay in Jac; `cloc` at every gate
No LMT data access	No real buses	Simulator is the default; demo never depends on LMT
Own map looks plainer than Google Maps	Judges read it as a sketch	Real stop coordinates, 3D buses, live rerouting and a turn-by-turn driver view; optional OpenStreetMap base layer
Camera QR scan unreliable in a dark hall	Demo stalls	Manual code entry fallback under every QR; bright screen mode
Real-time lag with polling	Counts look stale	2 s polling for the demo; WebSocket walkers if time allows
Venue Wi-Fi	App unreachable	Run locally on the laptop with phone on a hotspot
Wrong Sinhala or Tamil wording	Looks careless to local judges	Native-speaker review of every string before the demo
Seven extra features stretch the timeline	Core flow unfinished on demo day	Build demo mode and alerts first; drop spoken announcements and passenger incident reports if short of time

Assumptions

- The project is a competition prototype; no real payments or personal data beyond a username.
- Bus capacity is a configurable seat count per bus (default 40); standing passengers are not modelled.

- One booking per passenger per bus trip.
- Stop QR codes are shown on the admin/bus screen; a printed stop sign is a later option.
- Driver and conductor names are invented sample names.

Acceptance criteria

- [x] ~~Everything marked Today in section 0 demonstrated~~
- [x] ~~Every test for a feature built today (section 13) passes in `jac test`~~
- [x] ~~`cloc` report shows ≥ 85% Jac~~
- [x] ~~5-minute judge script runs end to end twice in a row without a restart~~
- [x] ~~App running on the demo laptop and backup recording ready~~

15. Team setup and GitHub workflow

Three developers share one GitHub repo. Dev A (Claude Code, Pro plan) owns the core: the graph model and every walker with real logic. Dev B and Dev C (Antigravity) own the passenger app and the admin/driver app. Dev A publishes walker contracts and working stubs in week 1, so B and C never wait on the core.

Roles

Developer	Tool	Owns	Folders
Dev A — core and integration	Claude Code (Pro)	Graph model, all walkers, routing and traffic engine, seat counting, scan rules, forecast, QR signing, demo engine, seed data, tests, CI, reviews and merges	`core/`, `tests/`, `scripts/`, `.github/`

Developer	Tool	Owns	Folders
Dev B — passenger app	Antigravity	Landing and login, Park & Ride flow (EV, accessible slots), 3D yard, combined ticket, bus map and booking, QR scanner, trip view and arrival alerts, Sinhala and Tamil, accessibility settings	`pages/`, `components/passenger/`, `i18n/`, `glue/`
Dev C — admin and driver	Antigravity	Admin Home map, bus account, next-stop panel and stop QR, scan feed, alerts, parking and charger tab, incident console, demo console and caption bar, driver view with turn-by-turn and incident button	`admin/`, `driver/`, `components/admin/`

Why Dev A gets the hard parts: the walkers hold every rule that can be subtly wrong — per-segment seat counting, the booking state machine, duplicate-proof scans, auto-cancel and mismatch checks, Dijkstra rerouting with the 1-minute rule, incident re-planning and the scripted demo. Claude Code works across the whole repo, runs the tests and can review B's and C's pull requests. The Pro plan's usage resets every few hours, so Dev A should spend it on these reasoning-heavy tasks and on reviews, not on styling.

Repo layout

```
parkride/
├── jac.toml                  # project config, npm deps, Tailwind
├── main.jac                  # entry: app() router, imports pages
├── AGENTS.md                 # rules for every coding agent (source of
truth)
├── CLAUDE.md                 # one line: @AGENTS.md
├── docs/
|   ├── sow.md                # export of this SOW
```

```
|   ├── contracts.md          # every walker: inputs, outputs, errors (owned
by Dev A)
|   ├── jac-reference.md      # condensed Jac reference for agents
|   └── demo-script.md        # the judge story, step by step
├── core/                     # Dev A
|   ├── models/               # nodes.jac, edges.jac
|   ├── walkers/              # parking, booking, scans, routing, traffic,
buses, admin, demo
|   ├── lib/                  # dijkstra, allocation, forecast, qr_sign,
clock
|   └── seed/                 # seed.jac + data/*.json (stops, roads, buses,
slots)
├── pages/                    # Dev B: passenger screens (*.cl.jac)
├── admin/                    # Dev C: admin screens (*.cl.jac)
├── driver/                   # Dev C: driver view (*.cl.jac)
├── components/
|   ├── shared/               # map renderer, QR view, buttons (changes need
2 reviews)
|   ├── passenger/            # Dev B
|   └── admin/                # Dev C
├── i18n/                     # en.json, si.json, ta.json
├── glue/                     # small .ts only: three.js loop, camera QR
scanner
├── tests/                    # test_*.jac (Dev A writes core tests; B and C
add UI flow tests)
├── scripts/jac_share.py      # counts lines by extension, fails under 85%
Jac
└── .github/
    ├── workflows/ci.yml      # jac check, jac test, jac_share on every PR
    ├── CODEOWNERS            # core/ @devA, pages/ @devB, admin/ driver/
@devC
    └── pull_request_template.md
```

How the three work together

- **Contracts first.** `docs/contracts.md` lists each walker's inputs, outputs and errors. Changing one needs a PR labelled `contract-change`, approved by Dev A and the developer who calls it.
- **Stubs in week 1.** Every walker exists from day 5, returning seed data, so B and C build real screens against real calls. Dev A then replaces stubs with logic without changing the contract.

- **Branches.** `main` is protected. Work on short branches named `a/F-21-ev-slots`, `b/park-flow`, `c/next-stop-panel`; open a PR at least daily; squash-merge after CI passes and one review.
- **Reviews.** Dev A reviews every B and C PR with Claude Code (`/review` or "review this PR against AGENTS.md and contracts.md"). B and C review each other's UI PRs.
- **Issues.** One GitHub issue per requirement ID (P-, B-, A-, F-), labelled `dev-a`, `dev-b` or `dev-c`, on one GitHub Project board.
- **CI on every PR.** Install Jaseci, run `jac check` on every `.jac` file, run `jac test`, and run `scripts/jac_share.py` (fail if Jac is under 85%).
- **One shared seed.** Everyone runs the same seed data, so screenshots and bugs match.

AGENTS.md (paste into the repo)

```
# Park & Ride — rules for every coding agent
- Before coding, read the SOW section for your task in docs/sow.md and the
walker contracts in docs/contracts.md.
- Language is Jac. New logic goes in .jac or .cl.jac. A .ts file is allowed
only in glue/, under 80 lines.
- Stay in your folders: core/ (Dev A), pages/ and components/passenger/ (Dev
B), admin/, driver/ and components/admin/ (Dev C).
- Never change a walker's inputs or outputs without a contract-change PR
approved by Dev A.
- Call the backend only with root spawn <walker>(...). No fetch or axios.
- Every visible string uses t("key") with entries in i18n/en.json, si.json
and ta.json.
- Accessibility: real buttons and labels, 44 px touch targets, 4.5:1
contrast, respect reduced motion.
- No Google Maps, no real payments, no secrets or API keys in code.
- Run jac check and jac test before every commit. Commit messages: feat(F-
21): add EV slot type.
- If a Jac syntax question comes up, check docs/jac-reference.md or the jac
mcp server; do not guess.
```

For Antigravity, add a workspace rule that says "Follow AGENTS.md in the repo root", and register `jac mcp` as an MCP server. For Claude Code, `CLAUDE.md` contains `@AGENTS.md`, and `claude mcp add jac -- jac mcp` adds the same Jac docs.

After the hackathon: week-by-week split (today's hour-by-hour plan is in section 0)

Week	Dev A — Claude Code	Dev B — Antigravity	Dev C — Antigravity
1	Repo, CI, AGENTS.md, graph model, seed data, contracts.md, stubs for every walker, auth roles	Landing pop-up, login and signup, layout shell, i18n setup	Admin shell and routing, admin login guard, driver view shell
2	Slot allocation, EV and accessible rules, TripPass with signed QR, parking lifecycle	Park & Ride flow, EV question, slot grid, 3D yard, combined ticket screen	Parking tab, charger use, overstay list
3	Road graph, Dijkstra plan_route, DemoClock, bus simulator, LiveFeed adapter	Own SVG city map, 3D bus markers, bus card	Admin Home map, bus list, bus account page
4	Per-segment seat counting, book_seat, process_scan, auto-cancel, mismatch, forecast, notify_arrival	Seat booking flow, QR scanner, trip view, arrival alerts	Next-stop panel, stop QR, scan feed, alert feed
5	set_traffic, report_incident, re-planning, reroute logging	Delay notices on the trip view	Traffic and incident console, driver turn-by-turn, report-incident button
6	Hardening: full test suite, Jac-share fixes, performance, reviews	Sinhala and Tamil, accessibility pass	Accessibility pass on admin and driver
7	run_demo engine and demo script	Passenger screens in demo mode	Demo console and caption bar, rehearsals

After the hackathon: prompts — Dev A (Claude Code)

Run in the repo root. Use plan mode first for each task, approve the plan, then let it build.

1. **Kickoff (week 1).** "Read docs/sow.md and AGENTS.md. Set up the Jac project with jac create --use client. Create core/models with every node and edge from SOW sections 5, 8A and 9.6. Write docs/contracts.md listing every walker in the SOW with typed inputs, outputs and error codes. Implement each walker as a stub that returns realistic seed data, write core/seed from the appendix, add CODEOWNERS, the PR template, scripts/jac_share.py and a GitHub Actions workflow running jac check, jac test and jac_share. Commit in small steps."
2. **Parking (week 2).** "Replace the parking stubs with real logic: recommend_slots using the SOW 9.1 score and zones, EV and accessible rules F-13 and F-21–F-23, book_trip_pass creating a TripPass with an HMAC-signed QR, mark_parked, release_slot and the 30-minute hold. Write jac tests for every rule first, then make them pass. Keep contracts unchanged."
3. **Routing (week 3).** "Implement plan_route (Dijkstra on travel minutes, SOW 9.6), the DemoClock node, move_bus with the no-U-turn and 1-minute-or-10% switch rules, and the LiveFeed interface with SimulatorFeed. Port the rules from the traffic prototype linked in section 5 and add a test for each self-check it lists."
4. **Booking and scans (week 4).** "Implement per-segment seat counting (9.2), book_seat with pickup and drop validation, process_scan with the rule table in 9.4 using scan_id for idempotency, depart_stop with auto-cancel and the alighting mismatch check (9.5), forecast_seats (8A) and notify_arrival (F-01–F-04). Think through race conditions when two scans arrive at once. Tests for every row of 9.4."
5. **Traffic and incidents (week 5).** "Implement set_traffic and report_incident (F-24–F-27): create Incident nodes, apply road levels, re-plan affected buses, log Rerouted / Kept route / Waiting / Moving again, expire unconfirmed incidents after 30 minutes."
6. **Hardening (week 6).** "Run the full suite and jac_share. Move any logic found in .ts or client files into walkers until Jac is at least 85%. Profile the 2-second refresh with 9 buses and 200 bookings. Review open PRs from Dev B and Dev C against AGENTS.md and contracts.md and leave comments."
7. **Demo engine (week 7).** "Implement run_demo (F-28–F-30): read docs/demo-script.md as a list of steps that call the real walkers, with pause, back, next and restart, and a reset to seed in under 5 seconds. Add an end-to-end test that plays the whole script twice."

After the hackathon: prompts — Dev B (Antigravity, passenger app)

1. **Week 1.** "Read AGENTS.md and docs/contracts.md. In pages/, build the landing pop-up with Park & Ride and Get into Bus, plus login and signup using jacLogin and jacSignup. Set up i18n with t('key') and en, si, ta files. Use Tailwind and the layout of the passenger screens in the design canvas."
2. **Week 2.** "Build the Park & Ride flow: terminal picker, vehicle type, EV question (Needs charging / Park only), accessible-slot checkbox, arrive and return times, three slot recommendations from recommend_slots, and the combined ticket screen with the QR from book_trip_pass. Add the 3D yard with @react-three/fiber in a small glue file, highlighting the booked slot."
3. **Week 3.** "Build the passenger bus map as an SVG from the road graph returned by get_map, with route lines, stops and bus markers that move every 2 seconds from get_buses. Tapping a bus opens the bus card with seats now and the forecast."
4. **Week 4.** "Build seat booking (pickup and drop, passed stops hidden), the camera QR scanner calling process_scan, and the trip view with stops, alerts two stops before the drop and at the drop, and the 'Did you get off?' prompt."
5. **Weeks 5–6.** "Show incident delays on the trip view. Then fill si.json and ta.json for every key, using Noto Sans Sinhala and Tamil, and do an accessibility pass: text size, high contrast, reduced motion, keyboard use, screen-reader labels."

After the hackathon: prompts — Dev C (Antigravity, admin and driver)

1. **Week 1.** "Read AGENTS.md and docs/contracts.md. In admin/, build the admin shell with tabs Home, Buses, Parking, Alerts and Demo console, guarded by the admin role. In driver/, build the driver view shell."
2. **Week 2.** "Build the Parking tab: per-terminal slot grid, EV charger use and the overstay list from the parking walkers."
3. **Week 3.** "Build the admin Home map (own SVG map from get_map, live buses from get_buses), the bus list, and the bus account page with driver and conductor names."
4. **Week 4.** "Build the next-stop panel from admin_next_stop: stop, ETA, getting off, bookings, walk-ins, seats left, and the stop QR. Add the live scan feed and the alert feed."
5. **Week 5.** "Build the traffic and incident console (presets, per-road level, incident list with confirm, edit and clear), and the driver view: heading-up map, turn-by-turn bar, new-route banner and a two-tap Report incident button calling report_incident. Match the traffic prototype linked in section 5."

6. **Weeks 6–7.** "Accessibility pass on admin and driver screens. Then build the demo console and caption bar that drive run_demo with Play, Pause, Back, Next and Restart."

Appendix A. Sample seed data

All values below are invented for the prototype and loaded by the `seed` walker.

Parking slots per terminal

Zones split each type roughly 30% front, 40% middle, 30% rear. *Occupied at start* is the demo's opening state.

Terminal	Code	Foot cycles	Motorcycles	Cars	Occupied at start
Makumbura	`MKB`	40	150	120	12 / 96 / 74
Kadawatha	`KDW`	40	120	90	9 / 70 / 51
Kesbewa	`KSB`	30	80	60	6 / 41 / 33
Pettah	`PTH`	20	60	40	11 / 52 / 37

EV and accessible slots (part of the car totals above): Makumbura 8 EV (2 × 50 kW DC, 6 × 22 kW AC), Kadawatha 6 EV (22 kW AC), Kesbewa 4 EV (7 kW AC), Pettah 4 EV (22 kW AC). Each terminal also has 2 accessible car slots in the front line.

Routes and stop order

Route	Code	Stops in order
Makumbura – Pettah	`R1`	Makumbura · Kottawa · Battaramulla · Borella · Town Hall · Pettah
Makumbura – Kadawatha	`R2`	Makumbura · Apeksha Hospital · Nawinna · Narahenpita · Kelaniya Campus · Kadawatha
Kesbewa – Pettah	`R3`	Kesbewa · Piliyandala · Werahera · Boralesgamuwa · Kohuwala · Havelock · Pettah

Buses (40 seats each)

Bus	Route	Plate	Driver	Conductor
MB-101	R1	NB-4101	Sunil Perera	Kasun Fernando
MB-102	R1	NB-4102	Nimal Jayasinghe	Tharindu Silva
MB-103	R1	NB-4103	Ruwan Dissanayake	Chamara Bandara
MB-201	R2	NB-4201	Mahesh Wickramasinghe	Dilshan Rathnayake
MB-202	R2	NB-4202	Ajith Kumara	Nuwan Herath
MB-203	R2	NB-4203	Prasanna Gunawardena	Isuru Karunaratne
MB-301	R3	NB-4301	Lalith Senanayake	Sahan Weerasinghe
MB-302	R3	NB-4302	Upul Rajapaksha	Janith Abeysekara
MB-303	R3	NB-4303	Chaminda Samarasinghe	Hasitha Mendis

Road network

34 junctions (the 17 stops plus junctions such as Maradana, Dematagoda, Rajagiriya, Nawala, Kirulapone, Nugegoda, Thalawathugoda, Pannipitiya, Malabe, Athurugiriya, Kaduwela, Peliyagoda and Kiribathgoda) and 53 roads. Lengths are straight-line distance × 1.25. The full list is in the traffic prototype and can be copied into the seed walker.

Sources

- Jac full-stack guide (jac-client)
- Jac getting started
- Lanka Metro SmartMetro live tracker
- Ministry of Transport — Metro Bus Digital Platform POC notice
- Google Maps Platform Terms of Service